Problema 1

1. Reduccion -> B de NOT

TRUE = $\lambda x.\ \lambda y.\ x$ devuelve 1er argumento

FALSE = $\lambda x.\ \lambda y.\ y$ devuelve 2do argumento

NOT = $\lambda p.\ p$ FALSE TRUE

NOT TRUE

= $(\lambda p.\ p$ FALSE TRUE$)\ (\lambda x.\ \lambda y.\ x)$

$\rightarrow_B (\lambda x.\ \lambda y.\ x)$ FALSE TRUE

$\rightarrow_B (\lambda y.$ FALSE$)$ TRUE

$\rightarrow_B$ FALSE

NOT FALSE

= $(\lambda p.\ p$ FALSE TRUE$)\ (\lambda x.\ \lambda y.\ y)$

$\rightarrow_B (\lambda x.\ \lambda y.\ y)$ FALSE TRUE

$\rightarrow_B (\lambda y.\ y)$ TRUE

$\rightarrow_B$ TRUE

2- Recursion y ciclos

Recursion: como las funciones λ no tienen nombre, se usa el combinador Y.

$Y = \lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$

$Y\ g \rightarrow_B g\ (Y\ g)$

FACT = Y $(\lambda f.\ \lambda n.$ IF (ISZERO n) 1 (MULT n (f (PRED n))))

Ciclos: no hay for ni while porque no hay variables mutables. Un ciclo se hace con recursion, pasando el contador y el acumulador como parametros: suma i n s = if i > n then s else suma (i+1) n (s+i)

3. Cuando si y no

SI: cuando se transforman datos y el procesamiento es paralelo, porque no hay estado compartido.

Ej: procesar millones de transacciones con map / filter / reduce

NO: cuando hay mucho estado que cambia rapido, tiempo real o poca memoria.

Ej: el motor de fisica de un videojuego o el firmware de un microcontrolador.